

PROYECTO: SISTEMA BÁSICO DE INVENTARIO

Instrucciones

Desarrolla una aplicación de escritorio en el lenguaje de programación de tu preferencia. Esta aplicación será utilizada para gestionar la base de datos SQLITE "InventarioBD.db" que se encuentra disponible en los archivos adjuntos del curso.

Diseño y presentación

La aplicación debe tener coherencia y ser visualmente atractiva. Se recomienda fuertemente que los colores primarios de su aplicación sean los colores de la Universidad de Sonora:

- **Azul Unison:** #00529e
- **Azul Oscuro Unison:** #015294
- **Dorado Unison:** #f8bb00
- **Dorado Oscuro Unison:** #d99e30

Utilice bordes redondeados de 4px en botones y en campos de entrada de texto. También puede basarse en algún estilo como Fluent Design o Material Design. Por último, el tipo de letra que debe utilizar en su aplicación debe ser Segoe UI para todos los textos.

Usuarios

Se debe agregar una tabla en la base de datos llamada "usuarios" con las columnas: nombre, password, fecha_hora_ultimo_inicio, rol. Donde la columna **rol** solo puede tener los valores "ADMIN", "PRODUCTOS" y "ALMACENES". Agrega los siguientes usuarios de base:

1. Nombre: ADMIN; Password: admin23; Rol: ADMIN
2. Nombre: PRODUCTOS; Password: productos19; Rol: PRODUCTOS
3. Nombre: ALMACENES; Password: almacenes11; Rol: ALMACENES

Es importante tener en cuenta que la contraseña debe estar encriptada en la base de datos, se sugiere un método de encriptación sencillo como MD5, pero se puede utilizar cualquier otro.

Productos y Almacenes

Se deben agregar nuevas columnas a las tablas *productos* y *almacenes*. Dichas columnas son las siguientes:

- `fecha_hora_creacion`: Debe guardar la fecha y hora de creación del producto. Establezca como valor predeterminado la fecha actual para todos los productos existentes.
- `fecha_hora_ultima_modificacion`: Debe guardar la fecha y hora de la última modificación que se le hizo a la información del producto.
- `ultimo_usuario_en_modificar`: Debe guardar el nombre del último usuario que modificó la información del producto.

Vista: Inicio de sesión

Al iniciar el programa debe mostrar una vista de inicio de sesión. Los datos para el inicio de sesión deben ser el *nombre del usuario* y su *contraseña*. También se debe mostrar un botón en la vista para iniciar sesión con los datos proporcionados. La FIGURA 1 muestra un boceto de la vista.

Es importante tener en cuenta que no debe existir una opción para registrar nuevos usuarios, debido a que la aplicación será utilizada de manera interna y contiene información sensible como lo son los datos de inventario del negocio. Tampoco deben existir opciones para recordar al usuario o mantener la sesión abierta.



FIGURA 1. Vista de inicio de sesión.

Cuando se ingresen credenciales válidas en los campos de inicio de sesión, la aplicación iniciará sesión, guardando al usuario y rol. Al momento de iniciar sesión se debe actualizar el valor de la columna fecha_hora_ultimo_inicio con la fecha y hora.

Vista: Inicio

Una vez se inicie sesión, se debe mostrar una vista de inicio dentro de la misma ventana, dicha vista de inicio debe tener el logo de la empresa, título de la aplicación, nombre del alumno que desarrolló la aplicación y una forma para navegar entre las distintas vistas (se sugiere un componente tipo NavBar). En la FIGURA 2 se muestra un boceto de la vista de inicio.



FIGURA 2. Vista de inicio.

Esta vista debe ser mostrada a todos los usuarios independientemente de su rol. También, cualquier usuario puede navegar a las distintas vistas.

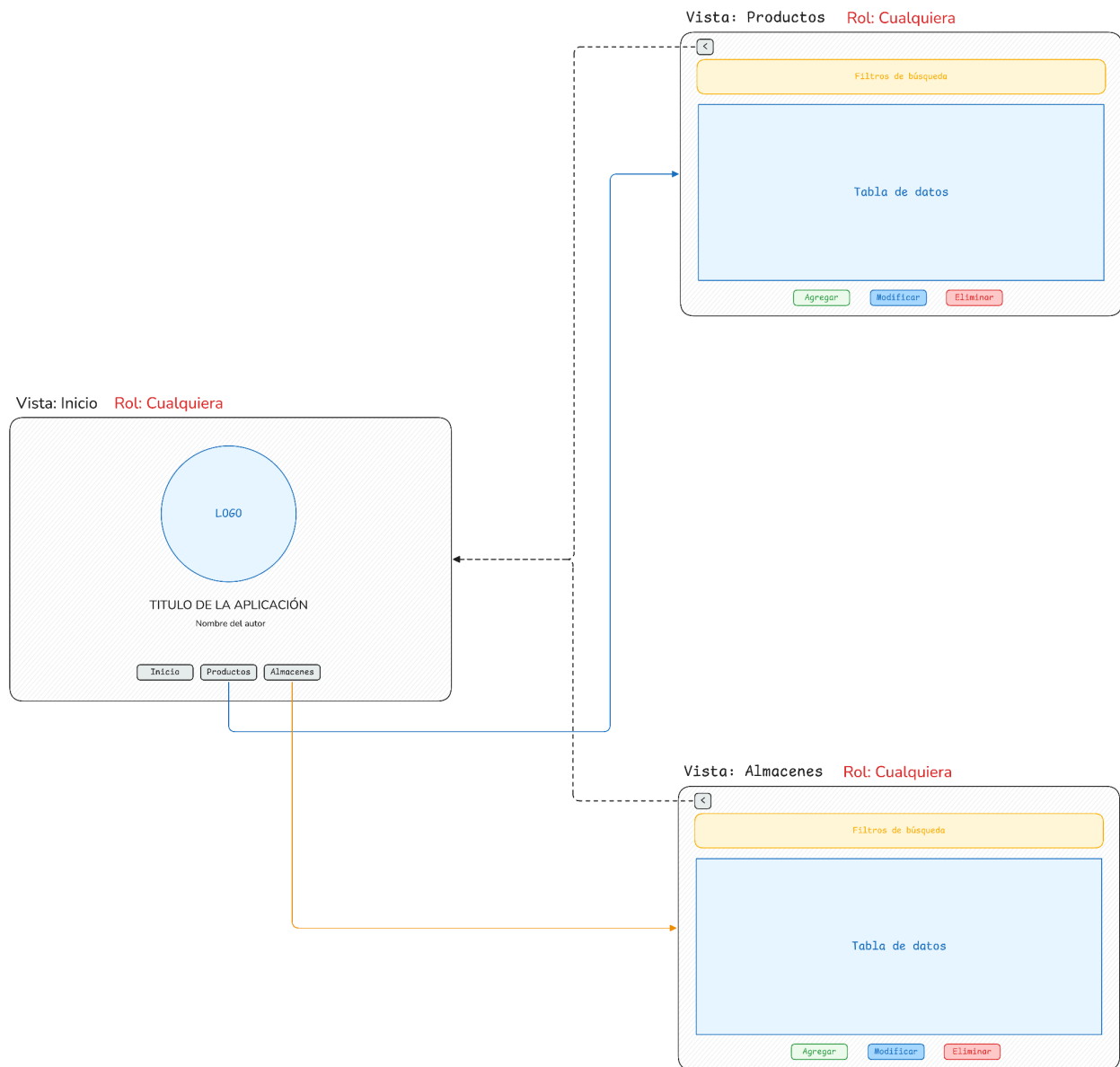


FIGURA 3. Navegación desde la vista Inicio.

Desde la vista Inicio se debe poder navegar a las vista Productos y la vista Almacenes (FIGURA 3). Toda la navegación debe ser dentro de una misma ventana. Si se abre una nueva ventana para mostrar estas vistas el proyecto tendrá un 0 de calificación final.

Vista: Productos

Esta vista se encarga de todo el manejo de los registros de la tabla productos. En la FIGURA 4 se muestra un boceto de la vista Productos.



FIGURA 4. Vista de productos.

Filtros de búsqueda

Esta sección de la vista debe contener filtros para todas las columnas de la tabla productos, las cuales deben mostrarse en la tabla de datos. El fin de estos filtros es que el usuario pueda filtrar productos según lo que necesite, ya sea nombres, fechas, cantidad, precio etc.

Es importante tomar en cuenta que para valores numéricos como la cantidad y el precio se deben aceptar rangos, no solo valores exactos.

Tabla de datos

En esta sección se debe mostrar una tabla con todas las columnas de la tabla productos de la base de datos, llena con los registros de la misma. Es importante tener en cuenta que para la columna *almacen*, **no** se debe mostrar el número del almacén, se debe mostrar el nombre al que corresponde ese id.

Agregar, Modificar y Eliminar

Se debe mostrar de alguna manera las opciones para Agregar, Modificar y Eliminar productos.

- Agregar: Esta opción debe permitir navegar a una vista con un formulario que cuente con todos los campos necesarios para agregar un nuevo producto.

- **Modificar:** Esta opción debe permitir navegar a una vista con un formulario que cuente con todos los campos necesarios para modificar la información de un producto existente, dichos campos deben tener pre-cargada la información del producto que se quiere modificar.
- **Eliminar:** Esta opción debe permitir eliminar un producto (registro) de la base de datos permanentemente, pero como método de seguridad debe mostrar algún diálogo de confirmación (puede ser una ventana emergente) que le pregunte al usuario si está seguro de eliminar dicho registro.

Estas opciones solo deben ser visibles para los usuarios con rol ADMIN o PRODUCTOS. En la FIGURA 5 se muestra un boceto de la vista y la estructura de la navegación.

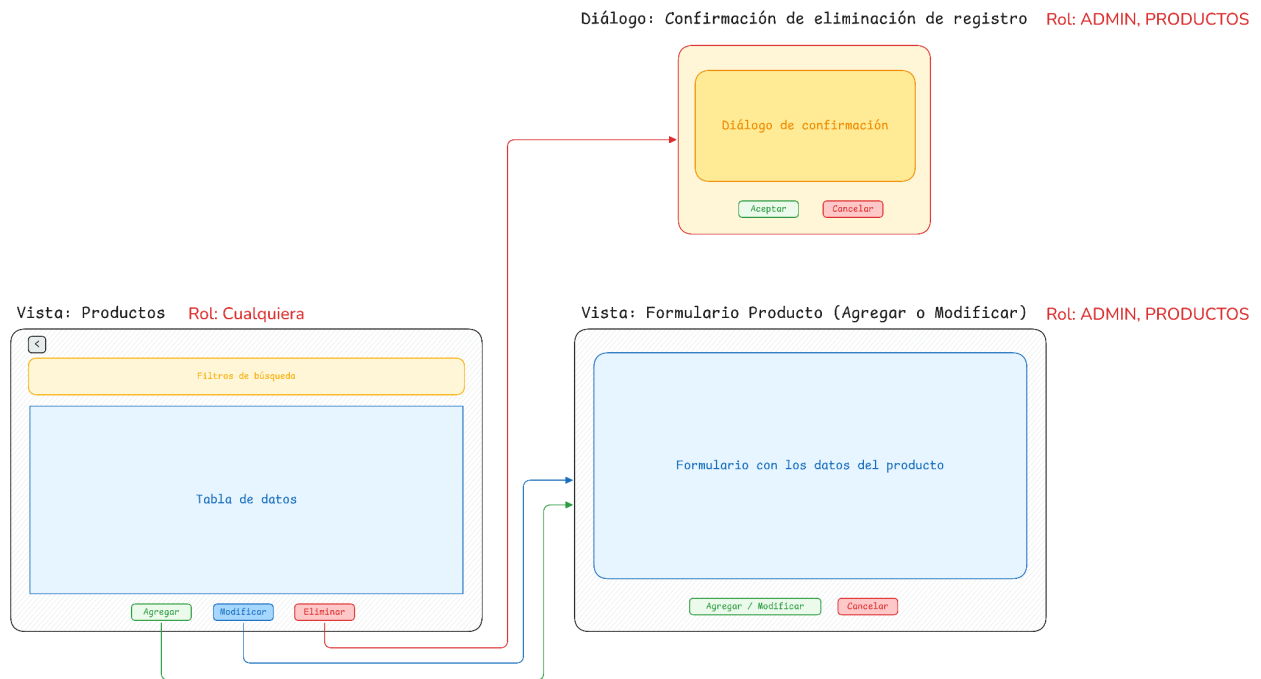


FIGURA 5. Navegación desde la vista Productos

Vista: Almacenes

Esta vista se encarga de todo el manejo de los registros de la tabla almacenes. En la FIGURA 6 se muestra un boceto de la vista Almacenes.



FIGURA 6. Vista Almacenes.

Filtros de búsqueda

Esta sección de la vista debe contener filtros para todas las columnas de la tabla almacenes, las cuales deben mostrarse en la tabla de datos. El fin de estos filtros es que el usuario pueda filtrar almacenes según lo que necesite, ya sea nombres, fechas, etc.

Tabla de datos

En esta sección se debe mostrar una tabla con todas las columnas de la tabla almacenes de la base de datos, llena con los registros de la misma.

Agregar, Modificar y Eliminar

Se debe mostrar de alguna manera las opciones para Agregar, Modificar y Eliminar almacenes.

- Agregar: Esta opción debe permitir navegar a una vista con un formulario que cuente con todos los campos necesarios para agregar un nuevo almacén.
- Modificar: Esta opción debe permitir navegar a una vista con un formulario que cuente con todos los campos necesarios para modificar la información de un almacén existente, dichos campos deben tener pre-cargada la información del almacen que se quiere modificar.

- Eliminar: Esta opción debe permitir eliminar un almacén (registro) de la base de datos permanentemente, pero como método de seguridad debe mostrar algún diálogo de confirmación (puede ser una ventana emergente) que le pregunte al usuario si está seguro de eliminar dicho registro.

Estas opciones solo deben ser visibles para los usuarios con rol ADMIN o ALMACENES. En la FIGURA 7 se muestra un boceto de la vista y la estructura de la navegación.

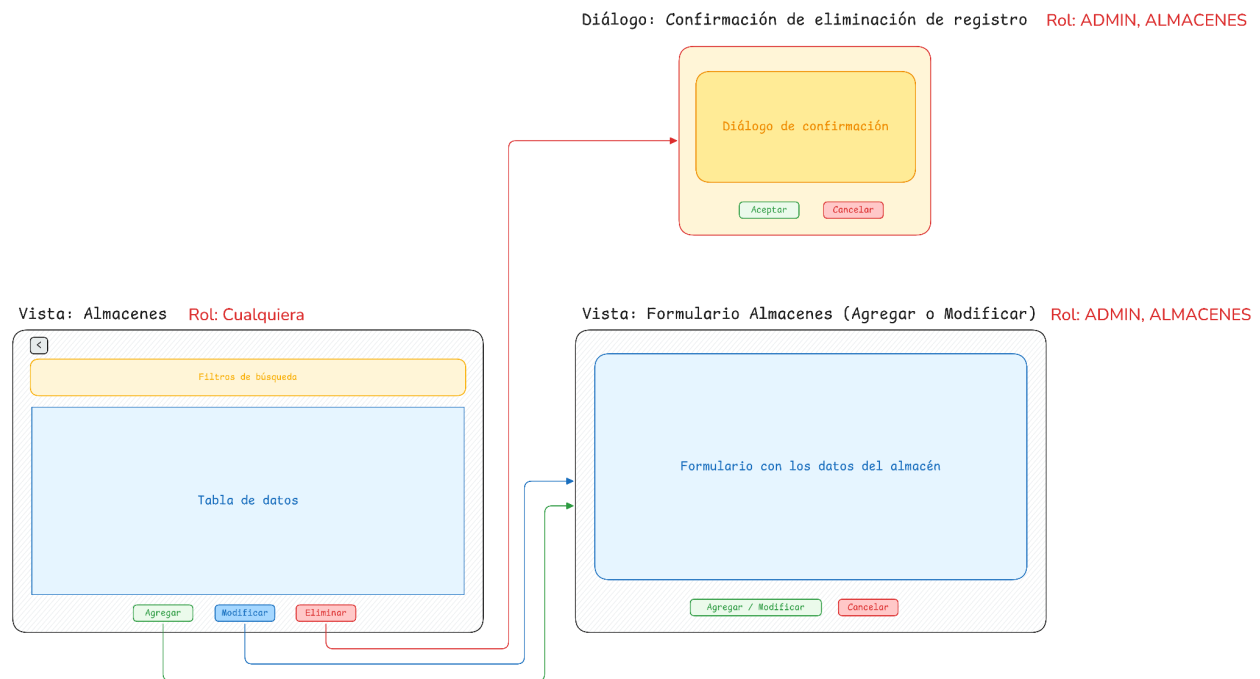


FIGURA 7. Navegación desde la vista de almacenes.

Mantenimiento y reingeniería

Un desarrollador creó un programa de inventario conforme a las especificaciones del archivo PDF adjunto, pero la aplicación presenta fallos y no está operativa. El cliente ha expresado su inconformidad y ha solicitado la corrección de los errores. Actualmente, la aplicación carece de la funcionalidad para agregar productos y presenta fallos críticos en la gestión de almacenes.

Instrucciones de Reingeniería:

- **Implementación de Pruebas Unitarias:** Se deben crear pruebas automatizadas para validar de forma individual los módulos de autenticación, el cálculo de inventarios y la lógica de encriptación de contraseñas.
- **Pruebas de Integración y Validación:** Realizar pruebas de flujo completo para asegurar que los datos se guarden correctamente en la base de datos SQLite y se reflejan en la interfaz de usuario.
- **Mejora del Diseño de las Interfaces:** Ajustar la UI para que cumpla estrictamente con la identidad visual (Azul #00529e y Dorado #f8bb00). Se debe asegurar que todos los botones, formularios y tablas mantengan una estética profesional y consistente.
- **Agrupación por paquetes:** Agrupa los archivos del proyecto en diferentes paquetes según su funcionalidad (interfaz, lógica de negocio, conexión a base de datos, modelos, etc.).
- **Documentación del Código:** Incluir comentarios detallados en el código fuente siguiendo estándares de documentación (como Javadoc o similares) para explicar la lógica de los métodos principales.
- **Manual de Software:** Generar un documento que incluya tanto el manual de usuario (guía de uso) como el manual técnico (arquitectura, dependencias y configuración de la base de datos).

Rúbrica de evaluación:

Criterio	Excelente (100%)	Suficiente (70%)	Insuficiente (40%)	Valor
Diseño y presentación	Aplica correctamente los colores institucionales, tipografía Segoe UI y border redondeados.	Cumple parcialmente con el diseño o presenta inconsistencias menores.	No sigue los lineamientos solicitados.	15%
Autenticación y roles	Login funcional con contraseñas encriptadas y gestión de roles (Admin, Productos, Almacenes)	El login funciona pero la encriptación o los roles son incompletos.	No hay encriptación o los roles no restringen el acceso.	15%
CRUD y SQLite	Las operaciones de Agregar, Modificar y Eliminar funcionan sin errores en todos los módulos.	Las operaciones funcionan pero presentan errores menores o falta de validación.	Fallas críticas al guardar o recuperar información	20%
Filtros de búsqueda	Búsquedas funcionales por columnas y rangos (fechas/precios) en tiempo real sin comprometer el rendimiento de la aplicación.	Filtros básicos implementados sin soporte para rangos complejos.	Filtros inexistentes o no funcionales.	10%
Navegación	Toda la aplicación se ejecuta en una sola ventana sin abrir procesos externos.	La navegación es funcional pero confusa o abre ventanas extra.	La navegación es deficiente y no respeta la regla de ventana única.	10%
Pruebas unitarias	Incluye un conjunto completo de pruebas que validan la lógica central del sistema	Pruebas parciales que no cubren todos los módulos críticos.	Ausencia total de pruebas automatizadas.	15%
Documentación	Código fuente debidamente documentado y manuales técnico/usuario completos.	Documentación básica o manuales con falta de detalle.	Sin documentación ni manuales de software.	15%

Documentación de los cambios

Durante el proceso de mantenimiento y reingeniería se encontrarán distintos problemas a resolver, todos estos problemas y las soluciones implementadas por el equipo deben estar debidamente documentados.

Uso de la inteligencia artificial

Se permite el uso de herramientas de inteligencia artificial para el desarrollo del proyecto, sin embargo, todo el uso debe estar debidamente documentado, desde prompts hasta las respuestas del modelo.

Es importante aclarar que la inteligencia artificial es una herramienta que debe apoyar al desarrollador en su trabajo, no sustituir el pensamiento crítico. La inteligencia artificial no debe determinar la dirección del desarrollo, debe ser guiada por el desarrollador y actuar como facilitadora del proceso. Si el proyecto muestra evidencia de ser generado en su totalidad o con más del 60% de IA no se aceptará.

Entregables

Para acreditar el curso se deben entregar lo siguiente:

Reporte

Se debe entregar un reporte con el siguiente contenido:

1. Portada.
2. Introducción. La introducción debe ser sobre el programa, ¿para qué sirve? ¿por qué es útil?, etc.
3. Funcionalidad del software. Aquí se debe describir la funcionalidad del software.
4. Implementación actual. Aquí describan de forma general el estado actual del software.
5. Errores y soluciones. Describe los errores encontrados en el software y el cómo los solucionaron hasta tener una versión funcional. Incluya las correcciones en el diseño y las pruebas unitarias que permitan identificar cualquier error futuro.
6. Conclusiones. Describe qué aprendiste al aplicar reingeniería, ¿fue complicado?, ¿qué parte se dificultó más?, ¿Qué parte se facilitó más?. Además, si se hizo uso de IA como herramienta, describa en media cuartilla como la IA lo ayudó o perjudicó, y si recomendarías la IA para hacer reingeniería .

GitHub

Enlace a un repositorio GitHub que debe contener todo el código fuente. Incluya un ejecutable (.exe o .msi) del programa listo para ejecutarse. SOLO UNO POR EQUIPO.

Presentación

Se debe realizar una presentación en vivo de la aplicación funcionando, dicha presentación puede o no estar acompañada de una presentación en PowerPoint, Canva o cualquier herramienta de su preferencia (siempre que no esté hecho con IA).

Se calificará lo siguiente:

- Diseño y presentación.
- Funcionalidad.
- Pruebas de escritorio.
- Conocimiento de la aplicación desarrollada.

Consideraciones

No se permitirá corregir código o arreglar pequeños errores durante la presentación ni volver a presentar durante la semana de ordinarios (van a extraordinario). Antes de presentar se debe tener todo listo y funcional, considera tener todo listo al menos dos días antes de la presentación para evitar problemas.